Parul® University
Vadodara, Gujarat

NAAC GRADE A++

Information and Communication Technology

Control Statements

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor(CSE), PIT
Parul University

1. Looping Statements.....1
2. Jump Statements 3
3. Summary of Key Concepts.....5
4. References.....5

3.3.1 Looping Statements

Looping in Java allows a set of instructions to be executed repeatedly based on a condition. It helps avoid code repetition and makes programs more efficient.

1. `for` Loop (Entry Controlled)

- **Definition**: The for loop is used when the number of iterations is known in advance.
- **Syntax**:

```
for (initialization; condition; update) {

   // loop body

}
```

➢ Example:

```
for (int i = 1; i <= 5; i++) {
   System.out.println("Count: " + i);
}
```

➢ Use Cases:

- Counting iterations
- Traversing arrays
- Running loops for a fixed number of times

2. while Loop (Entry Controlled)

- **Definition**: Used when the number of iterations is not known. The condition is evaluated **before** the loop body.
- **Syntax**:

```
while (condition) {

   // loop body

}
```

➢ **Example**:

```
int i = 1;
while (i <= 5) {
   System.out.println(i);
   i++;
}
```

➢ Use Cases:

- Waiting for user input
- Polling services
- Reading from a stream until EOF

3. do-while Loop (Exit Controlled)

- **Definition**: The loop body is executed **at least once**, and the condition is evaluated **after** the loop body.
- **Syntax**:

```
do {
   // loop body
} while (condition);
```

- Example:

```
int i = 1;
do {
   System.out.println(i);
   i++;
} while (i <= 5);
```

➢ **Use Cases**:

- Menu-driven programs
- User input validation
- Retry mechanisms

4. Nested Loops

- **Definition**: A loop inside another loop.
- **Example**:

```
for (int i = 1; i <= 3; i++) {
   for (int j = 1; j <= 2; j++) {
      System.out.println("i: " + i + ", j: " + j);
   }
}
```

➢ **Use Cases**:

- Matrix operations
- Pattern printing
- Multi-level iteration problems

3.4.1. Jump Statements

Jump statements are used to **control the flow of execution** by jumping out of loops, skipping iterations, or exiting methods.
There are **three main jump statements** in Java:

1. break Statement

- **Purpose**: Terminates the nearest enclosing loop or switch statement immediately.
- **Syntax**:

```
break;
```

- Example:

```
for (int i = 1; i <= 5; i++) {
   if (i == 3) break;
   System.out.println(i);
}
```

- Labeled break

  Used to break from nested loops or a specific outer loop.

  - Example:

```
outer:
for (int i = 1; i <= 3; i++) {
   for (int j = 1; j <= 3; j++) {
      if (j == 2) break outer;
      System.out.println("i=" + i + ", j=" + j);
   }
}
```

2. continue Statement

- Purpose: Skips the current iteration and continues with the next iteration of the loop.
- Syntax

```
continue;
```

- **Example**:

```
for (int i = 1; i <= 5; i++) {
   if (i == 3) continue;
   System.out.println(i);
}
```

Labeled continue

- Skips to the next iteration of the **outer loop** when nested loops are used.
- **Example**:

```
outer:
for (int i = 1; i <= 2; i++) {
   for (int j = 1; j <= 3; j++) {
      if (j == 2) continue outer;
      System.out.println("i=" + i + ", j=" + j);
   }
}
```

3. return Statement

- **Purpose**: Exits from the current method and optionally returns a value.
- **Syntax**:

```
return;       // no value (void method)
return value;  // with value (non-void method)
```

- **Example**:

```
public int square(int n) {
   return n * n;
}
```

Comparison Table

Statement	Purpose	Used In	Control Type
break	Terminates loop/switch	Loops, switch	Exit loop
continue	Skips current iteration	Loops	Skip iteration
return	Exits from method	Methods	Exit method

➢ Summary of Key Concepts:

- **Entry Controlled Loops:** The for and while loops execute code based on conditions evaluated before each iteration.
- **Exit Controlled Loops & Nested Loops:** The do-while loop evaluates the condition after the iteration. Nested loops involve one loop inside another, with specific use cases like multi-dimensional arrays. Infinite loops occur when a loop's termination condition is never met, requiring proper handling to avoid system overloads.
- **Jump Statements**: break exits loops, continue skips the current iteration, and return exits from methods. Labeled break/continue allows specifying which loop to control in nested loops.

➢References:

- **Oracle Java Documentation – Flow Control (Loops & Jump Statements)**
  https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html
- **GeeksforGeeks – Java Looping Statements (for, while, do-while)**
  https://www.geeksforgeeks.org/looping-statements-in-java/
- **W3Schools – Java Loops (for, while, do-while)**
  https://www.w3schools.com/java/java_for_loop.asp
- **TutorialsPoint – Java Control Flow (Loops and Jump Statements)**
  https://www.tutorialspoint.com/java/java_loop_control.htm
- **Java Complete Reference (Herbert Schildt) – Book**
  Especially Chapters on Loops, Nested Loops, and Jump Statements
- **Programiz – Java Loops and Jump Statements**
  https://www.programiz.com/java-programming/loop

Parul® University
Vadodara, Gujarat

NAAC GRADE A++

https://paruluniversity.ac.in/